Приклад: ***Розробка UML-моделі інформаційної системи 'Овочева база'***

Зміст

Оглавление

Зміст 1

Вступ 1

1. Загальна частина. Проектування автоматизованої інформаційної системи 2

1.1 Опис предметної області 2

1.2 Проблеми предметної області та пропозиції щодо автоматизації інформаційної системи 3

1.2.1 Опис проблем предметної області 3

1.2.3 Нефункціональні вимоги 3

2 Формування архітектури інформаційної системи 4

2.1 Концептуальна модель предметної області 4

2.1.1 Діаграма прецедентів 4

2.1.2 Діаграма станів 7

2.1.3 Діаграма класів 7

2.2 Логічна модель предметної області 9

2.2.1 Діаграма діяльності. 9

2.2.2 Діаграма послідовності 10

2.2.3 Діаграма класів 12

2.3 Фізична модель предметної області 13

2.3.1 Діаграма класів 14

2.3.3 Діаграма розгортання 15

2.4 Реалізація моделі в середовищі CASE-засобу 16

Вступ

Ця курсова робота присвячена питанням. Подібна технологія набуває все більшого поширення при створенні сучасного програмного забезпечення. Підходи і методи, які використовуються на різних етапах створення об’єктно-орієнтованих систем і є предметом обговорення в даному посібнику. Основна увага приділяється початковим етапам розробки: постановці завдання, формалізації предметної області, проектування структури систем з використанням об'єктно-орієнтованої методолгії. Розглядаються питання застосування об'єктів-орієнтованого аналізу, проектування, викладаються основи універсальної мови моделювання UML. Також наводиться інформація про тестування об'єктно- орієнтованих систем та огляд інструментальних засобів розробки. Посібник містить приклади практичного використання даних методологій.

Сучасні CASE-засоби охоплюють велику область підтримки численних технологій проектування 1С: від простих засобів аналізу і документування до повномасштабних засобів автоматизації, що покривають весь життєвий цикл. Найбільш трудомісткими етапами розробки 1С є етапи аналізу та проектування, у процесі яких CASE-засоби забезпечують якість прийнятих технічних рішень та підготовку проектної документації. При цьому велику роль відіграють методи візуального представлення інформації. Це передбачає побудову структурних чи інших діаграм у реальному масштабі часу, використання різній кольорової палітри, наскрізну перевірку синтаксичних правил. Графічні засоби моделювання предметної області дозволяють розробникам в наочному вигляді вивчати існуючу ІВ, перебудовувати їх у відповідність з поставленими цілями та наявними обмеженнями.

У розряд CASE-засобів потрапляють як відносно дешеві системи для персональних комп'ютерів з дуже обмеженими можливостями, так і дорогі системи для неоднорідних обчислювальних платформ і операційних середовищ. UML - уніфікована мова моделювання для опису, візуалізації та документування об'єктно-орієнтованих систем і бізнес-процесів в ході розробки програмних додатків. Докладно описуються базові поняття UML, необхідні для побудови об'єктно-орієнтованої моделі системи з використанням графічної нотації. Виклад супроводжується прикладами розробки окремих діаграм, які необхідні для подання інформаційної моделі системи. Мета електронної книги - допомогти програмістам освоїти нову методологію розробки корпоративних програмних додатків для подальшого застосування отриманих знань з використанням відповідних CASE-інструментів. Якщо спробувати охарактеризувати сучасний рівень розвитку комп'ютерних та інформаційних технологій, то перше, на що слід звернути увагу - це зростаюча складність не тільки окремих фізичних та програмних компонентів, але і лежать в основі цих технологій концепцій та ідей. Здається, ще зовсім недавно професійному програмістові було достатньо досконало володіти однією-двома мовами програму- вання, щоб розробляти серйозні програмні додатки. Вибір платформи та операційної системи, як правило, не був серйозною проблемою. А супровід програми, хоча і було пов'язане з об'єктивними труднощами, могло бути реалізовано простим додаванням або зміною коду вихідної програми.

Представлена курсова робота з теми "Розробка UML-моделі інформаційної системи "Овочева база"" розглядає процес створення програмного продукту від задуму до виконуваного коду. Для розробки використовуються UML-діаграми, які побудовані за допомогою популярного CASE-засобу StarUML.

1. Загальна частина. Проектування автоматизованої інформаційної системи

1.1 Опис предметної області

Метою даної курсової роботи є розробка моделі програми, призначеної для автоматизації роботи овочевої бази, яка дозволить значно підвищити продуктивність праці за рахунок автоматизації функцій, які раніше виконувалися вручну.

У проектуванні використовується мова UML. Система призначена для підтримки роботи невеликої овочевої бази, основним видом діяльності якої є продаж та закупівля продуктів (овочів). Так як на базі вибір овочів досить широкий, то часто дуже важко вручну, розглядаючи вітрини або вивчаючи каталоги, знайти спорідний продукт, внести інформацію про новоприбулих клієнтів.

Автоматизована інформаційної система:

— дозволяє автоматизувати замовлення;

— формує довідники необхідні для роботи: з замовником, постачальником,

товаром (продуктами);

— дозволяє автоматизувати створення документації підприємства;
— автоматизує складський облік.

1.2 Проблеми предметної області та пропозиції щодо автоматизації інформаційної системи

1.2.1 Опис проблем предметної області

Процес створення автоматизованих інформаційних систем (далі, АІС) Овочевої Бази - це сукупність робіт, що починаються формуванням вхідних вимог до цієї системи, а закінчуються введенням її в дію. Такий процес передбачає або способи індивідуального розроблення проектної документації, або способи індустріального розроблення із застосуванням засобів автоматизованого проектування. Сучасні засоби автоматизованого проектування поділяються на два типи:

— такі, що базуються на методах типового проектування;
— такі, що базуються на методах модельного проектування;

У свою чергу, типове проектування передбачає застосування типових проектних рішень (ТПР), пакетів прикладних програм (ППП) або орієнтоване на об'єкт проектування в цілому. ТПР і ППП будуються за компонентами, а об'єктний підхід охоплює всю систему, тобто АІС овочевої бази.

При типовому проектуванні постає потреба пристосовувати ("прив'язувати") ТПР або ППП до конкретного об'єкта управління. Такий спосіб організації АІС дає змогу створювати уніфіковані елементи АІС і мінімізувати витрати на них. При створенні комп'ютерних інформаційних систем обліку, як правило, використовуються методи типового проектування.

1.2.2 Функціональні вимоги

(процес створення діаграми прецедентів та взаємодій)

Метою створення АІС овочевої бази є вдосконалення системи бухгалтерського обліку на конкретному економічному об'єкті завдяки застосуванню засобів обчислювальної техніки.

Проектування АІС овочевої бази - тривалий, трудомісткий і динамічний процес, в якому на різних етапах беруть участь фахівці різних напрямків і кваліфікації. Одним із головних завдань управління розробленням є чіткий поділ і координація робіт за групами фахівців у часі для успішного закінчення проектних робіт у директивне встановлені терміни.

Організації, причетні до створення комп'ютерних інформаційних систем овочевої бази, за своїми юридичними повноваженнями поділяються на дві категорії: замовники і виконавці. Замовником може бути кожна господарська організація, що потребує розроблення і впровадження АІС. Виконавцями, як правило, є спеціалізовані проектні організації (науково-дослідні та проектні інститути, проектні бюро і т. ін.), пов'язані із замовником договірними зобов'язаннями.

Процес створення комп'ютерної інформаційної системи овочевої бази являє собою сукупність упорядкованих у часі, взаємопов'язаних, об'єднаних у стадії та етапи робіт, виконати які необхідно і достатньо для створення АІС.

1.2.3 Нефункціональні вимоги

Зручність використання − інтерфейс користувача повинен бути Windows-сумісним.

Надійність − система повинна бути в працездатному стані двадцять чотири години на добу сім днів на тиждень, час простою − не більше десяти процентів.

Продуктивність − система повинна підтримувати можливість подальшого поєднання з технологічним обладнанням, яке використовується у приймальні та на складі.

Безпека − система повинна дозволяти працівникам корегувати інформацію лише в межах своєї компетентності шляхом встановлення паролів щодо запобігання розголошення

комерційної таємниці.

Проектні обмеження:

— система не містить розрахунку фонду заробітної плати в повному обсязі;
— система не враховує механізми оподаткування;
— система не враховує вартість упаковки;

Спеціальна частина. Розробка канонічних uml-діаграм автоматизованої інформаційної системи у середовищі case-засобу.

2 Формування архітектури інформаційної системи

2.1 Концептуальна модель предметної області

Візуальне моделювання в UML можна представити як деякий процес ступеневого спуску від найбільш загальній і абстрактній концептуальної моделі вихідної системи до логічної, а потім і до фізичної моделі відповідної інформаційної системи. Для досягнення цих цілей з автоматизації Овочевої бази, **спочатку будується модель у формі діаграми прецедентів тобто варіантів використання**, яка описує функціональне призначення системи або, іншими словами, те, що система буде робити в процесі свого функціонування.

Таким чином, діаграма варіантів використання є вихідним концептуальним уявленням або концептуальною моделлю системи в процесі її проектування і розробки.

Далі слід врахувати, що кожен прецедент системи має певну поведінку, може знаходитись в певних станах, а також переходити з одного стану в інший. Тобто, він моделює всі зміни станів певного варіанту використання як його реакцію на зовнішні впливи.

В разі чого, для специфікації функціональності прецеденту, що має складну модель поведінки, програмну систему з автоматизації ремонтно-будівельної компанії представляється з точки зору теорії кінцевих автоматів, тобто у **формі діаграми станів**.

2.1.1 Діаграма прецедентів

У результаті аналізу вимог виділяються такі актори:

1 "Замовник":
— Замовляє товар на складі;

2 "Працівник складського приміщення":
— Стежить за товаром (продуктами) на складі;
— Стежить за місцями на складі та товаром в них;
— Відповідає за строки зберігання товару та зміни їх у разі необхідності;

3 "Знищувач"
— Стежить за товаром, який буде знищено або знищується;

4 "Постачальник"
— Постачає продукти (товар) на овочеву базу;

5 "Секретар"
— Формує складовий асортимент товару и редагує його;

6 "Бухгалтер"
— Стежить за фінансами підприємства та їх своєчасний прихід;
— Формує знижки.

Виходячи з потреб дійових осіб, виділяються наступні варіанти використання з переліком специфікацій:

1 "Складське приміщення":
— Короткий опис - приміщення, де зберігається товари;
— Основний потік подій - починає виконуватись, коли товар приходить на склад;

— Альтернативні потоки - відсутні.

— Передумови - передбачає наявність товару, місця де його будуть зберігати та слідкувати за його станом.

— Постумови - при успішному виконанні товар відправляють до замовника. Інакше товар буде знищено.

2 "Замовлення":

— Короткий опис - замовляння товару;

— Основний потік подій - починає виконуватись коли проходить замовник;

— Альтернативні потоки - відсутній.

— Передумови - Прихід замовника.

— Постумови - при успішному виконанні замовник отримує товар в своє повне розпорядження.

3 "Товар":

— Короткий опис - товар який повинен прийти на склад;

— Основний потік подій - починає виконуватись коли організація замовляє у постачальника, при цьому беручи дані про товар із асортименту постачальника.

— Альтернативні потоки - відсутній.

— Передумови - коли із сформованих довідників Списку постачальників та довідника Асортименту закуплених товарів надійдуть дані;

— Постумови - при успішному виконанні закуплені товари відправляють на склад;

4 "Асортимент"

— Короткий опис - довідник в якому зберігається інформація про товар який є у постачальників. Ці товари можуть бути закуплені на даний період;

— Основний потік подій - від постачальника, який знаходиться у списку постачальників з якими організація веде справи надходить інформація: які товари можуть надати (формується довідник з товарами які можна закупити на даний проміжок часу).

— Альтернативні потоки -відсутні;

— Передумови - формується на підставі каталогу постачальників товарів.

— Постумови - при успішному виконанні передасть дані про товар який можна закупити в результаті чого товар буде закуплений.

5 "Термін зберігання":

— Короткий опис - Містить дані про товар: стан та час зберігання на складі;

— Основний потік подій - формується довідник про товар.

— Альтернативні потоки - відсутні.

— Передумови - немає;

— Постумови - при успішному виконанні передає дані про зберігання товару та час його зберігання;

6 "Бухгалтерія":

— Короткий опис - Містить дані про рахунки клієнтів, виплату за товар, кредитні рахунки;

— Основний потік подій - формується довідник про грошові операції на підприємстві.

— Альтернативні потоки - відсутні.

— Передумови - передбачає наявність замовлення.

— Постумови -немає;

7 "Знижки":

— Короткий опис - Містить дані про знижки які будуть представленні постійним клієнтам;

— Основний потік подій - формується довідник про знижки які підприємство буде давати своїм постійним клієнтам.

— Альтернативні потоки - відсутні.

— Передумови - немає;

— Постумови - при успішному виконанні передає замовник отримує знижки.

8 "місце":

— Короткий опис - Містить дані про зберігання товару його стан та спосіб зберігання;

— Основний потік подій - формується довідник про товар який на даний момент зберігається на складі.

— Альтернативні потоки - відсутні.

— Передумови - немає;

— Постумови - при успішному виконанні передає на склад характеристику товару та місце його перебування.

9 "Акт списання": ·

— Короткий опис - Довідник, який містить дані про товар, що не відповідає стандартам або терміну зберігання.

— Основний потік подій - відсутній.

— Альтернативні потоки - товар який не відповідає стандартам якості підлягає знищенню.

— Передумови - формується довідник про товар який був знищений;

— Постумови - при успішному виконанні товар знищується та передається документ про витрати в бухгалтерію.

Поведінка акторів і прецедентів системи відображаються їх відносинами, які показані на рисунку 2.1.

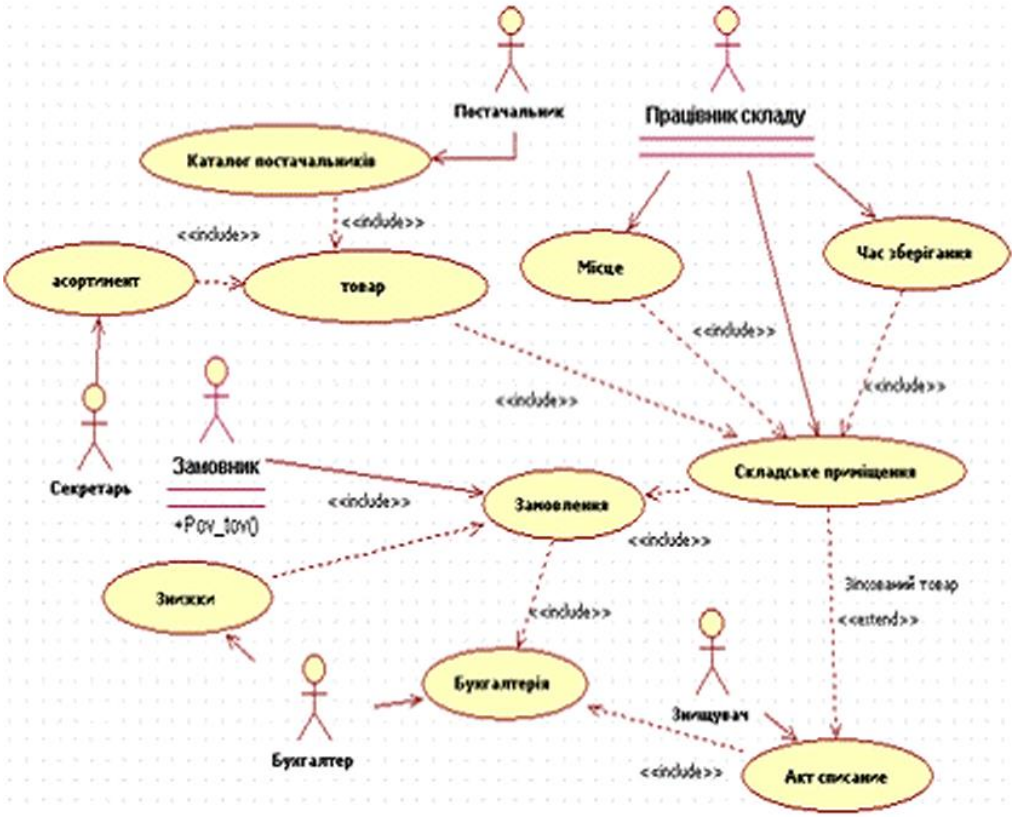


Рисунок 2.1 - Діаграма прецедентів.

2.1.2 Діаграма станів

Діаграма станів на концептуальному рівні дозволяє описувати можливі послідовності станів і переходів окремого прецеденту, які характеризують його динамічну поведінку. Тому, в рамках завдання автоматизації овочевої бази, виділяються прецеденти, що володіють неоднозначною поведінкою.

Спочатку йде пошук товарів на складі. При знаходженні, товар рахується і порівнюється з початковими даними. Якщо дані не співпадають, йде запит на його списання. Списаний товар перевіряють при знаходженні недостачі програма йде на вихід. Якщо ні, запит поступає в бухгалтерію, де витягуються дані про продаж товару; при знаходженні недостачі програма йде на вихід. Якщо по документам дані не співпадають винуватця чекає догана або штраф на суму недостачі.

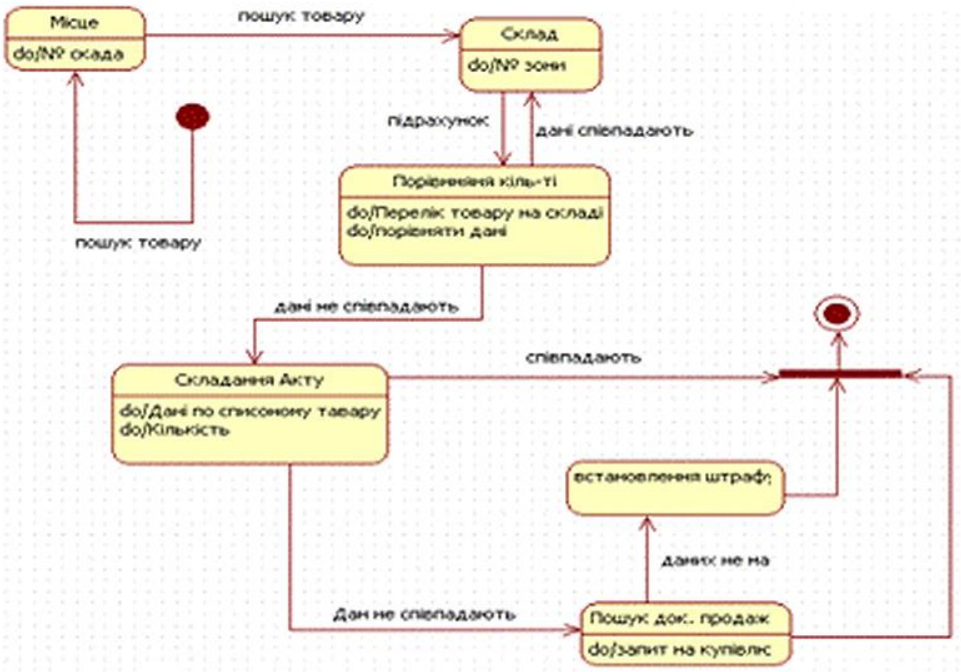


Рисунок 2.2 - Діаграма стану.

2.1.3 Діаграма класів

На концептуальному рівні класи використовуються в процесі аналізу предметної області для складання словника системи, що розробляється. При розробці даної форми діаграми класів виникають наступні основні завдання.

— виділення класів і визначення їх внутрішньої структури на основі аналізу функціональних вимог до автоматизованої системи.

— встановлення взаємозв'язків між класами, що забезпечують ефективність функціонування системи.

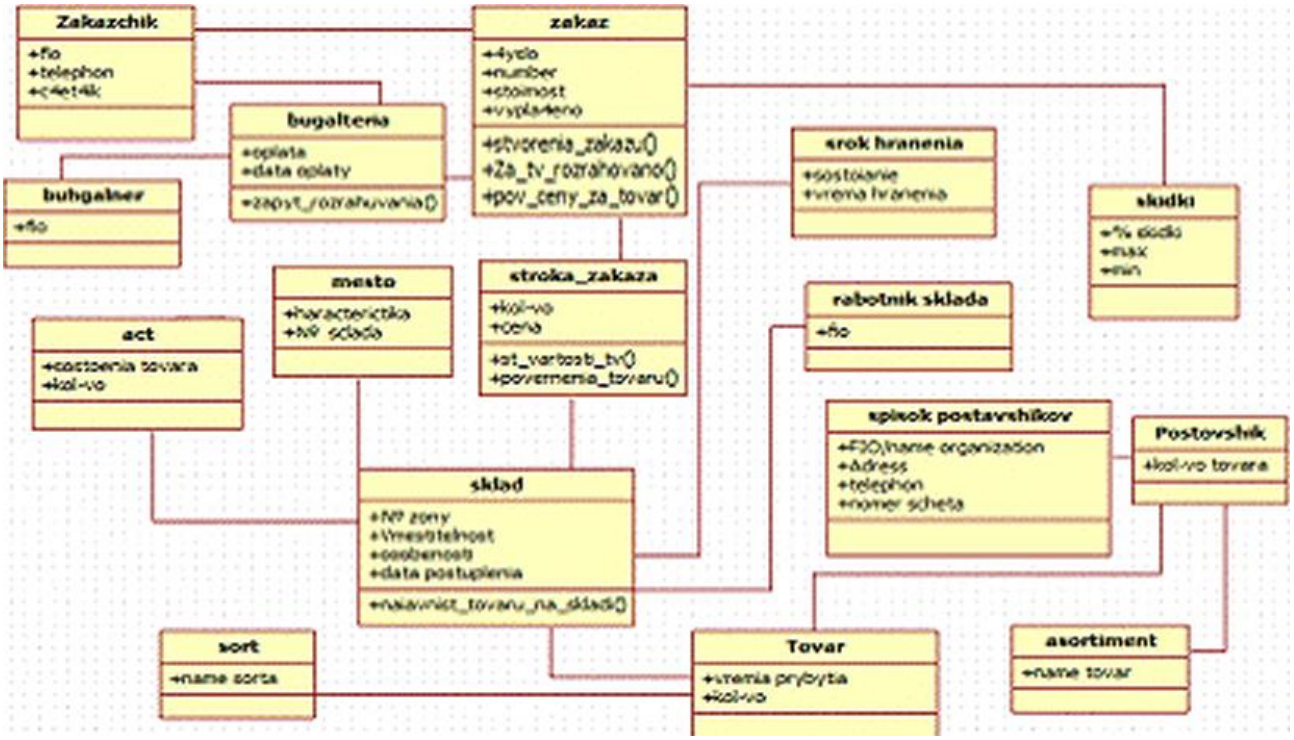


Рисунок 2.3 - Діаграма класів.

Замовник (zakazchik).

1. FIO - містить дані про замовника: Прізвище Ім’я По-батькові.
2. Telephone - містить дані які дозволяють зв’язатися з замовником.
3. Lichylnyk - рахує кількість замовлень зробленими цією людиною при наявності n-ї кількості замовлень переводиться із замовника в постійні клієнти.

Бухгалтерія(bugalteria):

1. Oplata - Містить дані про нарахування грошей за надані послуги.
2. data oplaty - дата оплати товару

Замовлення (zakaz):

1. Chyslo - дата прийому замовлення.
2. Number - номер замовлення.
3. Stoimost - Ціна замовлення.
4. Vyplacheno - Скільки грошей переказано на рахунок бази.

Рядок замовлення (stroka_zakaza)

1. kol-vo - Скільки було замовлено товару.
2. Cena - Ціна за кількість замовленого.

Складське примішення (sklad)

1. № zony - номер зони, знаходиться товар.
2. Vmestitelnost - кількість товару.
3. Osobenosti - особливості товару.
4. data postuplenia - дата коли товар прибув на склад.

Товар (Tovar)

1. vremia prybytia - дата прибуття товару.
2. kol-vo - кількість товару.

Асортимент (Asortiment)

name tovar - назва товару.

Постачальник (Postovshik)

fio/name ordonization - дані про постачальника або організацію.

Каталог постачальників (spisok postavshikov)

1. FIO/name organization - дані про постачальника або організацію

2. Telephon - номер телефона/факса/пейджера за яким можна зв’язатися з постачальником.

3. nomer scheta - номер рахунку на який поступають гроші за товар.

Час зберігання (srok hranenia)

1. sostoianie - містить опис товару в момент його прибуття на склад.
2. vrema hranenia - містить дані про час зберігання товару на складі.

Працівник складу (rabotnik sklada)

fio - Дані про працівника.

Бухгалтер (Buhgalner)

fio - дані про бухгалтера.

Акт списання (act)

1. sostoenia tovara - містить дані про стан товару після його перебування на складі деякий час.
2. kol-vo - кількість цього товару.

Місце (Mesto)

1. haracterictika - містить дані про товар.
2. № sclada - номер складу, де перебуває товар.

Сорт товару (Sort)

Name sorta - назва сорту продута.

Секретар (Sekretar)

FIO - інформація про секретаря.

Знищувач (Znyshuvach)

FIO - інформація про знищувача.

2.2 Логічна модель предметної області

розробка логічної моделі системи у вигляді діаграми класів

Оскільки логічна модель містить абстракції, які вже можуть бути незрозумілі експертам предметної області − ця модель служить для уточнення інформації про предметну область у тому вигляді, який зручний для подальшої реалізації інформаційної системи. Таким чином, логічна модель дозволяє повністю задати структуру даних, проте без "прив'язки" до конкретної платформі реалізації. Такий опис, з одного боку, виходить компактніше, ніж фізична модель і дозволяє поглянути на схему даних в цілому, без зайвих деталей. З іншого боку, така специфікація може бути в подальшому реалізована за допомогою різних мов програмування.

2.2.1 Діаграма діяльності.

При моделюванні поведінки проектованої або аналізованої системи виникає необхідність не тільки уявити процес зміни її станів, але і деталізувати особливості алгоритмічної і логічної реалізації виконуваних системою операцій. Традиційно для цієї мети використовувалися блок-схеми або структурні схеми алгоритмів.

Для моделювання процесу виконання операцій в UML використовуються діаграми діяльності. Кожна така схема акцентує увагу на послідовності виконання певних дій або елементарних операцій, які в сукупності призводять до отримання бажаного результату. Причому перехід в наступний стан спрацьовує тільки при завершені цієї операції в попередньому стані.

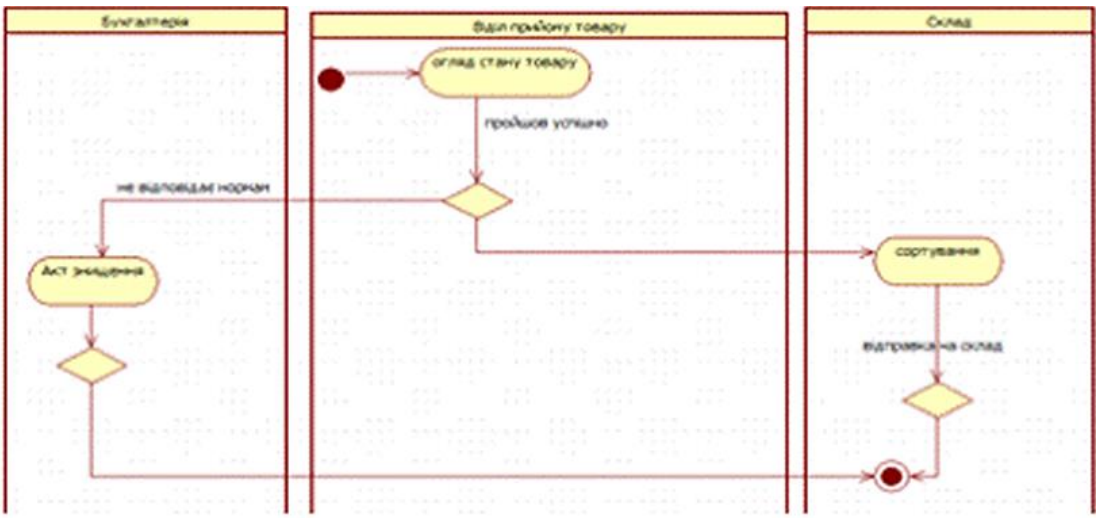


Рисунок 2.4 - Діаграма діяльності

Для уточнення концептуальної діаграми класів, яка була розглянута на рисунку 2.3, побудуємо діаграму, яка відображає сценарій "Прихід товару на склад".

На діаграмі діяльності на рисунку 2.4 відображається логіка або послідовність переходу від однієї діяльності до іншої, при цьому увага фіксується на результаті діяльності. Сам же результат може призвести до зміни стану системи або повернення деякого значення.

Продукти (товар) поступає до відділу приймання товару. Там товар проходить огляд. При успішному проходженні товар відправляється на склад, де сортується, розраховується кількість и т. д. У протилежному випадку товар знищується, а дані про знищення надходять в бухгалтерію, де заносяться до відповідного документу.

2.2.2 Діаграма послідовності

У мові UML взаємодія елементів розглядається в інформаційному аспекті їх взаємодії, тобто взаємодіючі об'єкти обмінюються між собою деякою інформацією. При цьому інформація приймає форму закінчених повідомлень. Іншими словами, хоча повідомлення і має інформаційний зміст, воно набуває додаткової властивості надавати направлений вплив на свого одержувача.

Ключовим моментом для діаграми послідовності є динаміка взаємодії об'єктів у часі. При цьому на діаграмі послідовності зображуються тільки об'єкти, які безпосередньо беруть участь у взаємодії, і ніякі статичні зв'язки з іншими об'єктами не візуалізуються.

Оскільки ініціатором виникнення ланцюжка подій в більшості випадків є деяка зовнішня сутність, то для виконання даної функції при вирішенні задачі автоматизації овочевої бази вибирається головний актор "Працівник складу", описаний раніше і наведений на діаграмі прецедентів на рисунку 2.1. Послідовність провокується "Працівником складу", коли завершено окремий вид робіт по даному об'єкту. При цьому, послідовність вступає у взаємодію з класами "Sklad", "Stoka hranenia", "Act".

Під час взаємодії формуються операції, які належать наступним класам:

"Sklad"

Peresmotr tovara() - Працівник складу приходить на склад и проводить огляд товару, який провів на складі деякий час ;

"Sklad"

Smena sroka hr. Tovara() - Після перегляду товару змінюється час зберігання.

"Srok hranenia"

peresmotr sroka hranenia() - якщо товар не відповідає вимогам його знову переглядають и перезаносять, вписуючи новий час зберігання.

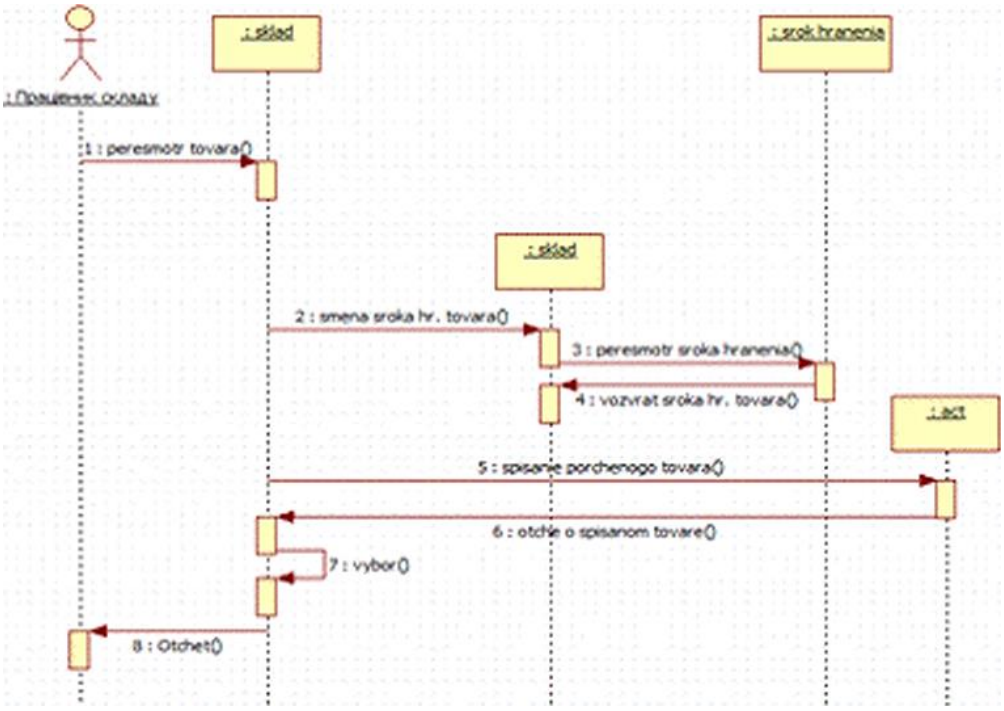


Рисунок 2.5 - Діаграма послідовності

"sklad"

vozvrat sroka hr. Tovara() - Товар якому змінили час зберігання знову повертається на склад.

"act"

spisanie porchenogo tovara() - Якщо товар не відповідає вимогам і після перегляду його заносять до акту знищення.

"sklad"

otchet o spisanom tovare() - відправляється звіт про товар, який був знищений.

Vybor() - товар знову переглядається и перераховується.

"Працівник складу"

Otchet() - виводиться кінцевий звіт. Діаграма, на якій показані взаємодії об'єктів, впорядкованих за часом їх прояви, представлена на рисунку 2.5.

2.2.3 Діаграма кооперації

Діаграми кооперації використовуються для характеристики динаміки поведінки систем, хоча час в явному вигляді в них відсутній.

Цей тип діаграм дозволяє описати взаємодії об'єктів, абстрагуючись від послідовності передачі повідомлень. На діаграмі в компактному вигляді відбиваються всі прийняті і передані повідомлення певного об'єкта і типи цих повідомлень.

В якості центрального об'єкту при побудові UML-моделі інформаційної системи овочевої бази обирається об’єкт класу "Zakaz", який приймає наступні повідомлення:

Stvorenia_zakazu ()

- короткий опис - створюється замовлення;
- ініціатор - актор "Замовник";
- вхідні параметри - id_zk (час виконання);
- результат - список товарів замовлення.

В якості центрального об'єкту при побудові UML-моделі інформаційної системи овочевої бази обирається об’єкт класу "Zakaz", який генерує наступні повідомлення:

st_vartosti_tv ()

— короткий опис - створює вартість товару;
— ініціатор - анонімний об'єкт класу "Zakaz";
— вхідні параметри - id_zr (ціна n-ї к-ті товару);
— результат - створення ціни.

na_rozrahunok ()

— короткий опис - передає дані про надходження грошей в касу;
— ініціатор - анонімний об'єкт класу "Zakaz";
— вхідні параметри - id_buh (Розраховано за товар);
— результат - розрахунок за замовлений товар.

Для передачі деяких повідомлень необхідно виконання операцій, що не стосуються центрального класу "Zakaz", але являються невід'ємною часткою його поведінки:

naiavnist_tovaru_na_skladi():

— короткий опис - перевірка на складі потрібної к-ті товару;
— ініціатор - анонімний об'єкт класу "Zakaz";
— власник - об’єкт класу "Stroka_zakaza";
— вхідні параметри - відсутні.
— результат - новий запис.

Порядок виникнення повідомлень, які відображають взаємодії наведених об'єктів, зазначений на діаграмі кооперації на рисунку 2.6.

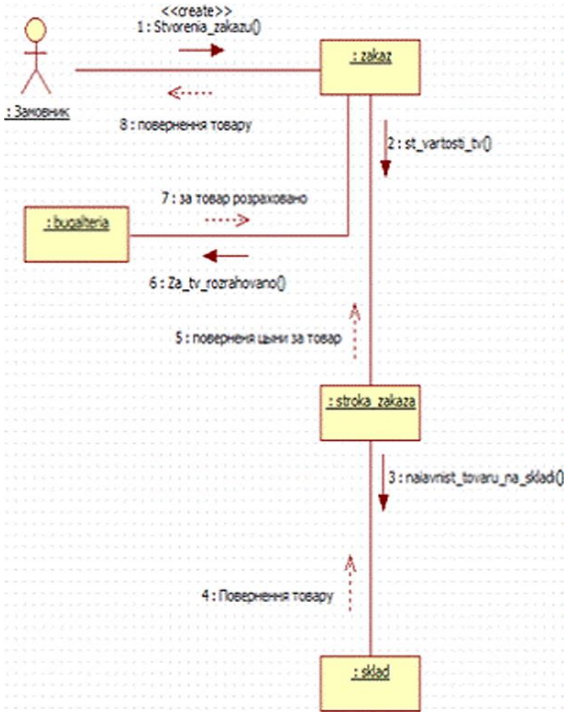


Рисунок 2.6 - Діаграма кооперації.

2.2.3 Діаграма класів

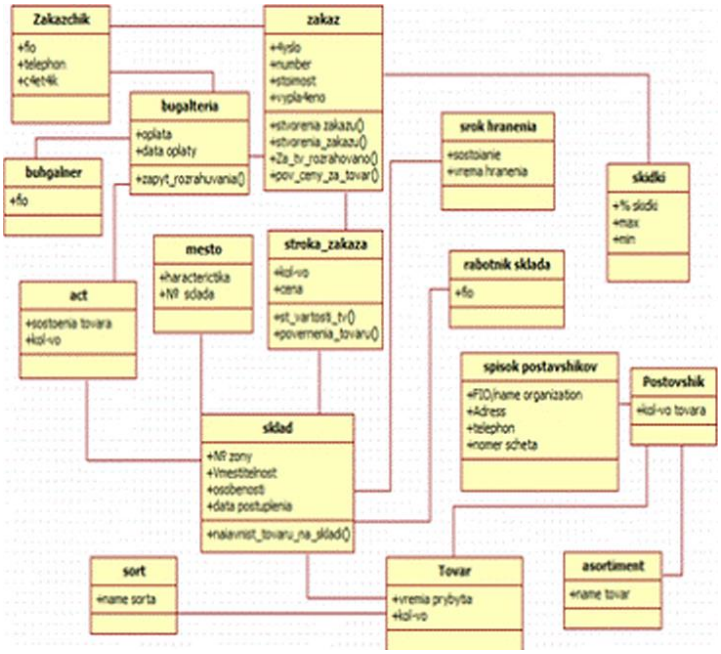


Рисунок 2.7 –Діаграма класів (Приклад 1).

Класи також пов'язані між собою різними зв'язками. наприклад:

клас leafCategory успадковує від класу Category;
клас compositeCategory може агрегувати об'єкти класу Category;
а клас leafCategory має навігаційну асоціацію з класом Product_Information (тобто, класи leafCategory посилаються на список об'єктів Product_Information).

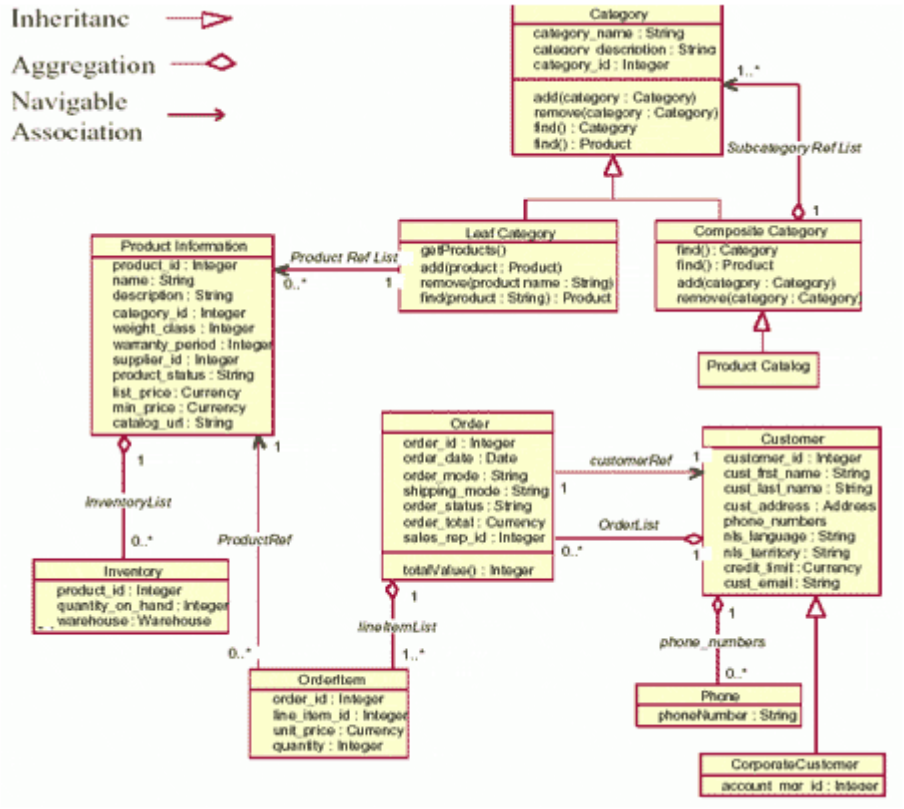


Рисунок 2.7 – UML-діаграма додатку "Оперативний каталог" (Online Catalog) (Приклад 2).

2.3 Фізична модель предметної області

Фізична модель інформаційної системи визначає способи розміщення даних в середовищі зберігання і способи доступу до цих даних, які підтримуються на фізичному рівні. Іншими словами, у мові UML фізична модель відображає компонентний склад проектованої системи з точки зору її реалізації на деякій технічній базі та обчислювальних платформах конкретних виробників.

2.3.1 Діаграма класів

Фізична модель діаграми класів є описом структури даних в термінах платформи реалізації − конкретної СУБД. Вона фактично є діаграмним поданням частини програмного коду, що визначає схему даних. Модель класів на фізичному рівні містить інформацію про різні деталі реалізації − індекси, ключі, типи атрибутів, межі видимості, які визначені в термінах цільової мови програмування C #.

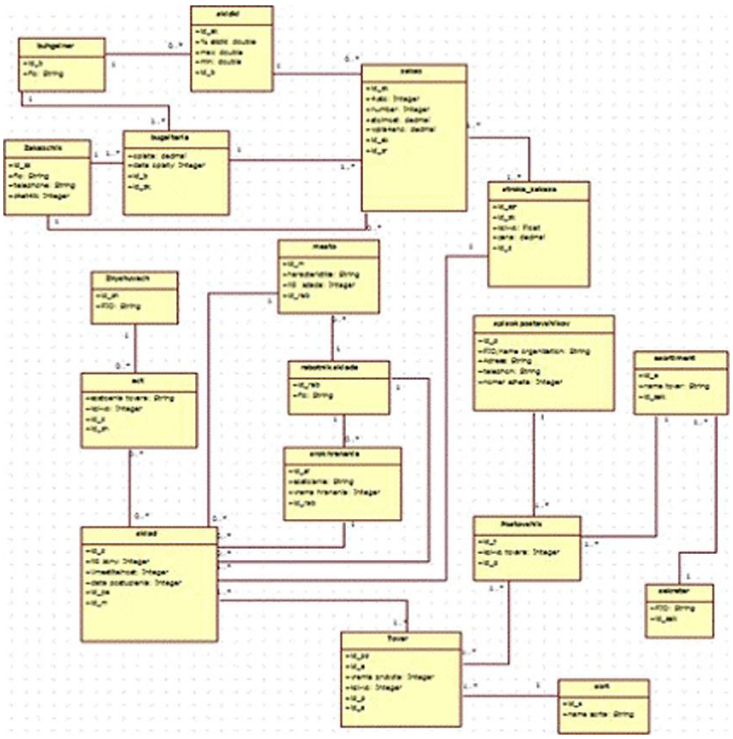


Рисунок 2.8 - Діаграма класів.

2.3.2 Діаграма компонентів

Діаграми компонентів застосовуються для моделювання статичного вигляду системи з точки зору реалізації. Сюди відноситься моделювання фізичних сутностей, розгорнутих у вузлі, таких як виконувані програми, бібліотеки, таблиці, файли і документи. По суті, діаграми компонентів − це не що інше, як діаграми класів, сфокусовані на системних компонентах.

Представлена на рисунку 2.9 діаграма компонентів містить у собі набір компонентів розв'язуваної задачі із встановленими для них відносинами.

Головна програма я містить у собі такі класи: "Buhgalteria", "otdel postashikov", "sklad", "Priemnia".

— "Priemnia"- має класи "Zakazchik", "zakaz";
— "sklad" - має класи "srok hranenia", "mesto", "Tovar";
— "Buhgalteria" - має класи "Act";
— "otdel postashikov"-має класи "asortiment", "Postovshik", "spisok postavshikov",

"Tovar".

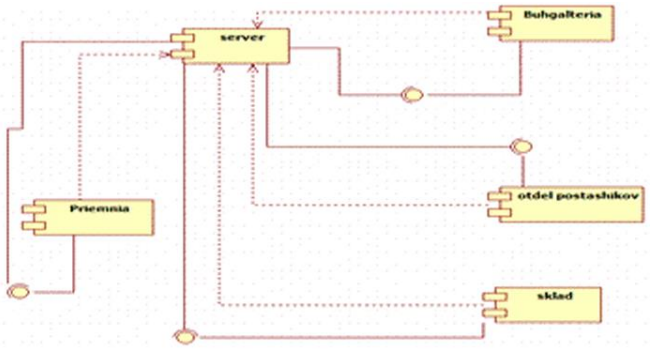


Рисунок 2.9 - Діаграма компонентів.

2.3.3 Діаграма розгортання

Фізичне подання програмної системи не може бути повним, якщо відсутня інформація про те, на якій платформі і на яких обчислювальних засобах вона реалізована. Якщо розробляється програма, що виконується локально на комп'ютері користувача і не використовує периферійних пристроїв і ресурсів, то в розробці додаткових діаграм немає необхідності. Інакше, для представлення загальної конфігурації і топології розподіленої програмної системи, створюються фізично існуючі елементи.

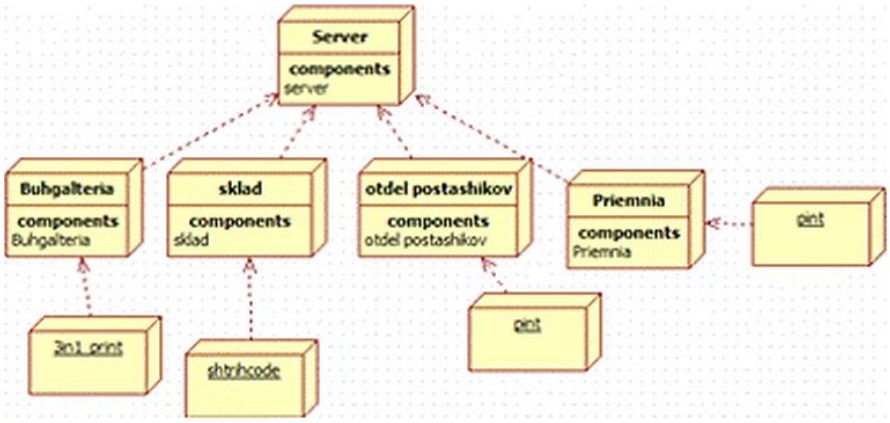


Рисунок 2.10 - Діаграма розгортання.

Головним комп’ютером (Server), на якому зберігаються дані з усіх підлеглих комп’ютерів, які розміщені в різних відділах овочевої бази.

1) комп’ютер, який знаходиться в бухгалтерії передає дані на Server.

. 3in1_print - для полегшення роботи працівникам бухгалтерії (роздрукування потрібної інформації).

2) комп’ютер, який знаходиться в складському приміщенні, веде складський облік. Передає дані на Server.

Shtrihcode - для полегшення роботи працівникам складу (занесення інформації про товар)

3) postashikov - комп’ютер, який знаходиться у відділі роботи з постачальниками, містить у собі дані про постачальника. Передає дані на Server.

Print - для полегшення роботи працівникам відділу роботи з постачальниками (роздрукування потрібної інформації).- комп’ютер, який знаходиться у відділі роботи з

замовниками, містить у собі дані про замовника, а також замовлений товар. Передає дані на Server.

Print - для полегшення роботи працівникам відділу роботи з замовниками (роздрукування потрібної інформації).

2.4 Реалізація моделі в середовищі CASE-засобу

Одним з найбільш важливих властивостей програми StarUml є можливість генерації програмного коду на декількох мовах програмування, яка може бути використана розробником після побудови моделі. Для цієї мети в середовищі StarUml присутній досить великий вибір мов програмування і схем баз даних.

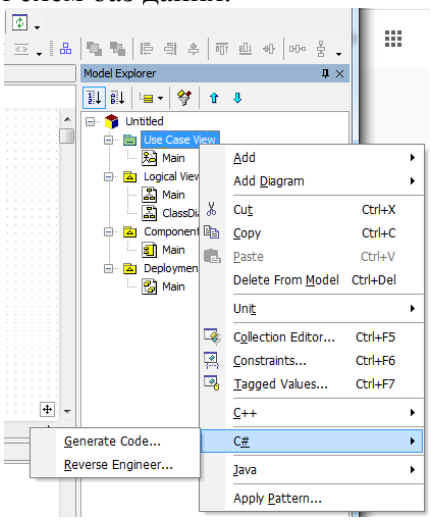


Рисунок 2.11 - StarUml, можливість генерації програмного коду

Використовуючи інформацію, що міститься в моделі проекту, генератор кодів на C# формує файли заголовків і файли реалізацій класів. Створювана структура програми може бути уточнена шляхом прямого програмування мовою C #. При повторній генерації внесені зміни зберігаються. Базуючись на обраних властивостях генерації, StarUml генерує наступні елементи коду:

а) для кожного модуля:
- файл заголовка та файл реалізації для модуля;
- #include директиви, явно чи неявно визначені в моделі.

б) для кожного класу:
- визначення класу;
- оголошення атрибутів і відносин;
- get і set функції для доступу до атрибутів;
- оголошення для стандартних операцій із заготовками визначень;
- оголошення для операцій, визначених користувачем.

Всі поля документування із специфікацій діаграм переносяться до коду як коментарі. Отриманий код наведено у додатку Б.

Висновки

Робота овочевої бази автоматизована. Проведено повний і детальний опис предметної області, виявлені межі розроблюваної системи.

Діаграми різних видів показкють систему з різних точок зору. В результаті розроблена модель відображає структуру задачі, що автоматизується, її дійових осіб, автоматизованих

функцій, інших сутностей, сценаріїв реалізації операцій і станів сутностей. Модель використовується для визначення вимог до системи, що розробляється, та виявлення всіх зв'язків між її частинами, що беруть участь у вирішенні конкретних завдань.

Результати моделювання в CASE StarUml легко можуть бути перетворені в документацію відповідну промисловим стандартам розробки програмних систем. Повний і детальний опис предметної області зручно проводити в CASE засобі, що підтримує універсальну мову моделювання UML, яка добре сприймається експертами предметної області і не вимагає від них ніякої спеціальної підготовки для розуміння поданих ним на розгляд моделей.

Список літератури

1. Крег Ларман, Застосування UML 2.0 і шаблонів проектування. 3-є вид. - М.: "Вільямс", 2006. - 736 с.

2. Джозеф Шмуллер, Освоюй самостійно UML 2 за 24 години. Практичний посібник. - М.: "Вільямс", 2005. - 416 с.

3. Грейда Буч, Джеймс Рамбо, Айвар Джекобсон, Мова UML. Керівництво користувача. 2. - М., СПб.: ДМК Пресс, Пітер, 2004. - 432 с.

4. Буч Г., Якобсон А., Рамбо Дж., UML. Класика CS. 2-е вид. / Пер. з англ.; Під загальною редакцією проф. С. Орлова - СПб.: Пітер, 2006. - 736 с.

5. Леоненко А.В. Об'єктно-орієнтований аналіз та проектування з використанням UML і IBM Rational Rose - БІНОМ. Лабораторія знань, Інтернет-університет інформаційних технологій - ІНТУІТ.ру, 2006

6. Грекул В.І., Денищенко Г.М., Коровкін Н.Л. Проектування інформаційних систем - Інтернет-університет інформаційних технологій - ІНТУІТ.ру, 2008

7. С. Орлов, Технології розробки програмного забезпечення. Підручник - СПб.: Пітер, 2002. - 464 с.

8. Лешек А. Мацяшек, Аналіз та проектування інформаційних систем за допомогою UML 2.0. 3 вид.: - М.: Вільямс, 2004. - 816с.

9. Кендалл Скотт, UML. Основні концепції: - М.: Вільямс, 2002. - 144с.

10.Боггс, UML і StarUml: - М.: Лорі, 2007. - 286с.

11. Национальный Открытый Университет "ИНТУИТ": www.intuit.ru
Александр Леоненков Лекция 5. Элементы графической нотации диаграммы классов https://www.intuit.ru/studies/courses/32/32/lecture/1008

12. - Простые стратегии для сложных данных: объектно-реляционная технология Oracle9i Источник: Geoff Lee и др., An Oracle Technical White Paper, май 2002 Перевод Алексея Резниченко http://citforum.ru/database/oracle/oracle-one/